

Clustering

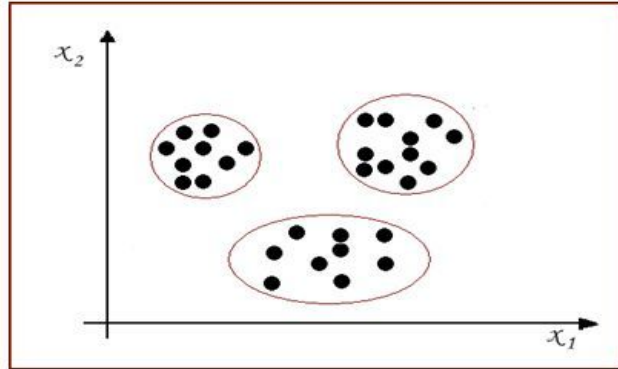
Agenda

- What is Clustering?
- Clustering vs. Classification
- K-means Algorithm
 - Initialization Trap
 - Inertia and Elbow Method
- Hierarchical Algorithm
 - Dendrogram

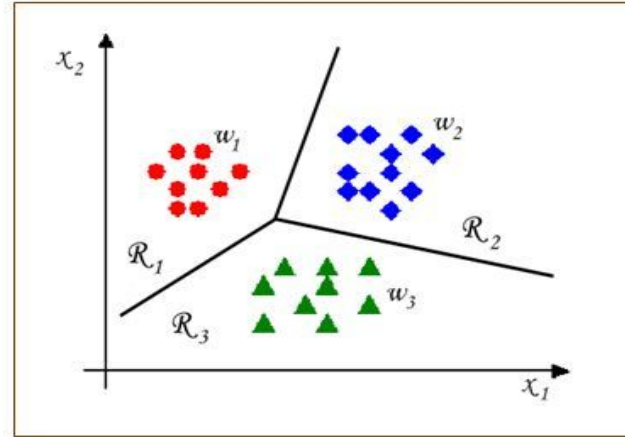
Clustering

- Unsupervised Learning : No target to be predicted
- Automatically discovering natural grouping in data.
- Clustering can be helpful as a data analysis activity in order to learn more about the problem domain, so-called ***pattern discovery*** or knowledge discovery.
- Examples:
 - Market Segmentation
 - News Grouping
 - Social Network Analysis

Clustering vs. Classification



Given some patterns, the goal is to discover the underlying structure (categories) in the data based on inter-pattern similarities



Given some training patterns from each class, the goal is to construct decision boundaries or to partition the feature space

Clustering

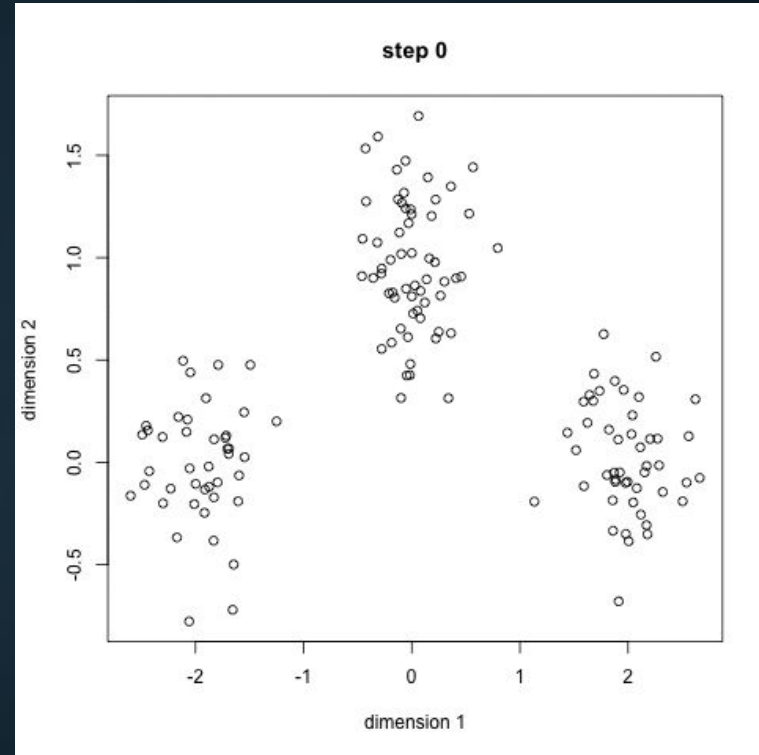
- Many algorithms *use similarity or distance measures* between examples to discover dense regions of observations.
- As such, it is often good practice to *scale data* prior to use clustering algorithms.
- The scikit-learn library provides a suite of different clustering algorithms to choose from, you can check here: [Clustering@Sklearn](#)
- Clustering is an *unsupervised learning* technique, so it is hard to evaluate the quality of the output of any given method.

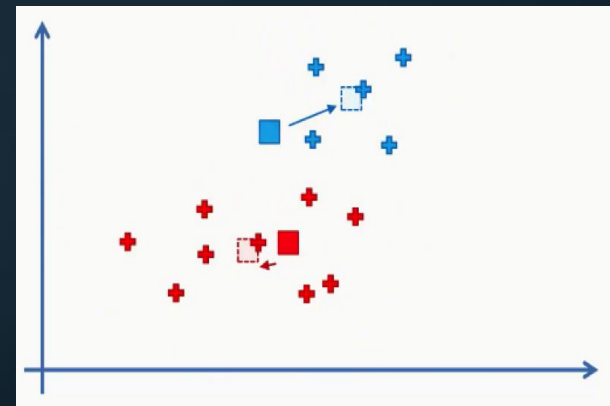
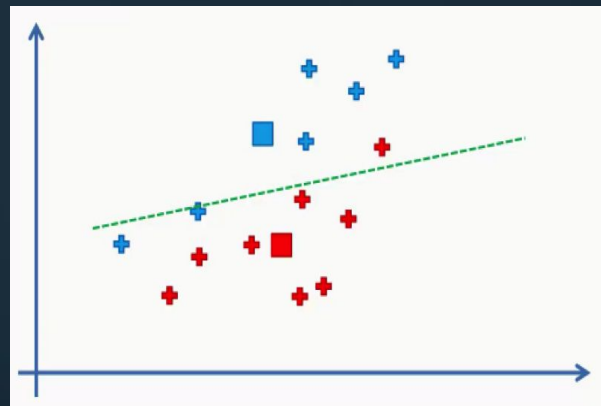
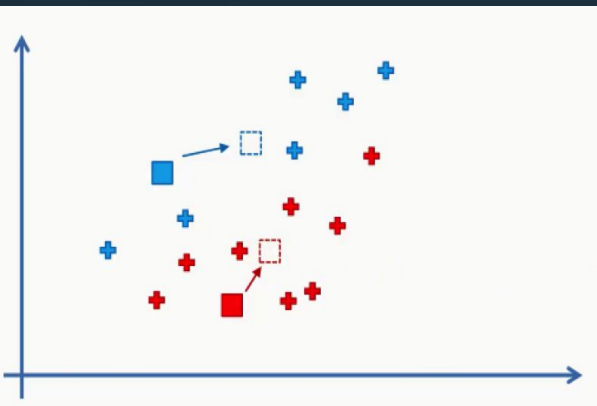
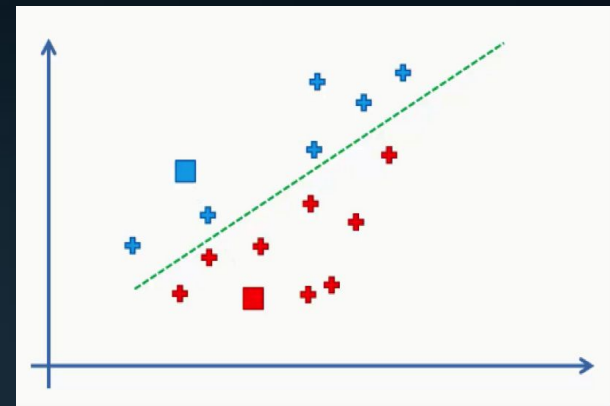
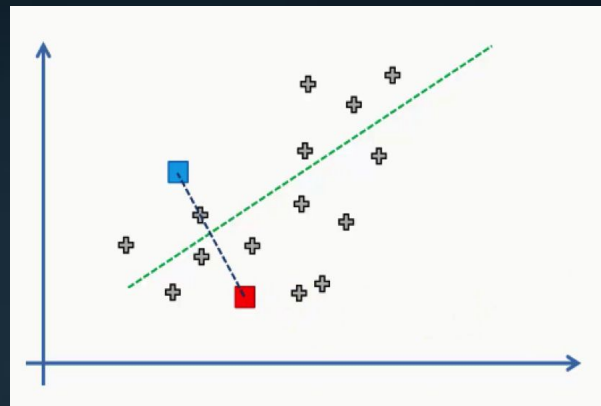
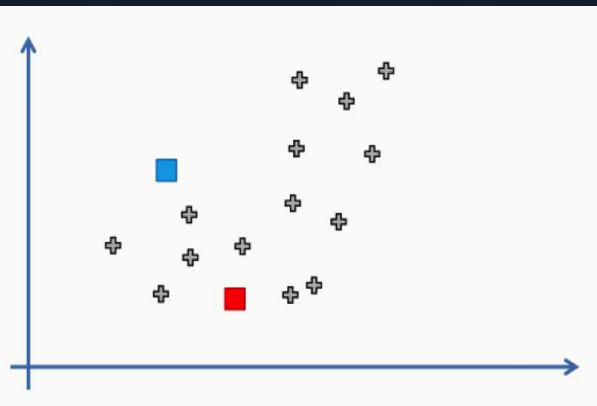
K-means Algorithm

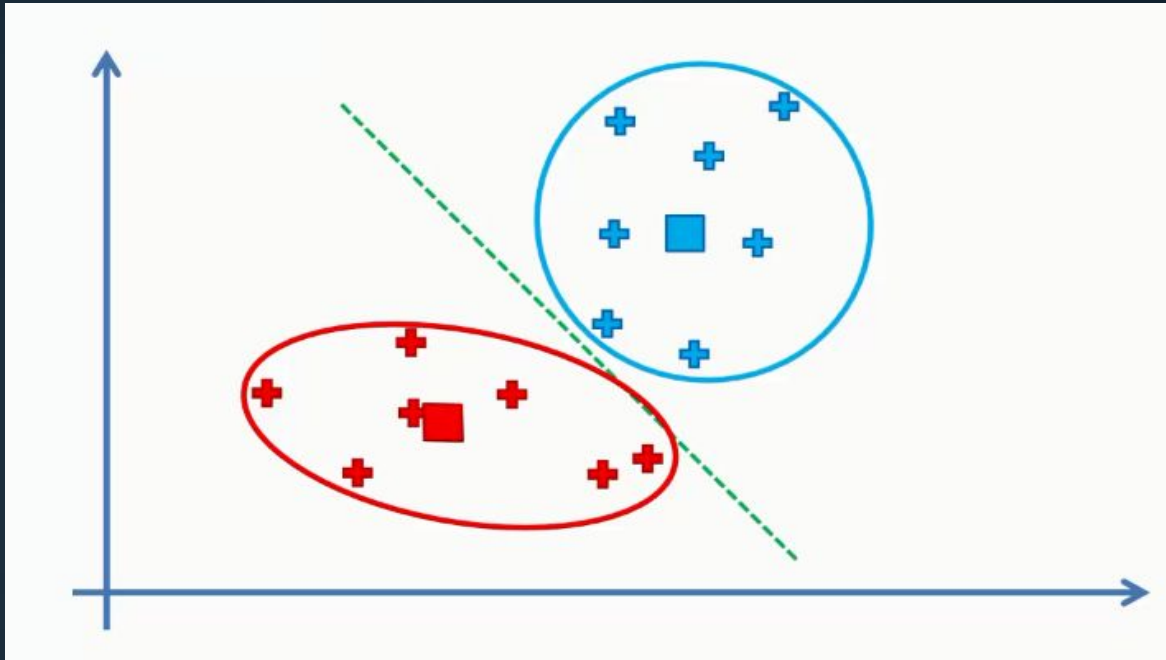
K-means Clustering Algorithm

- *Assigning examples to clusters in an effort to minimize the variance within each cluster.*

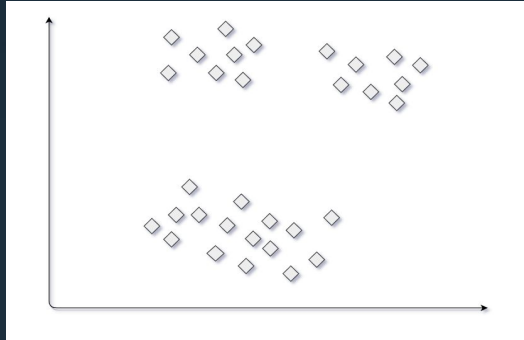
1. Choose the number k of clusters
2. Select at random k points, the centroids
3. Assign each data point to the closest centroid,
so we have k clusters
→ *Euclidean distance is generally used.*
4. Compute and place the new centroid of each cluster
5. Repeat steps 3, 4 till no change occurs



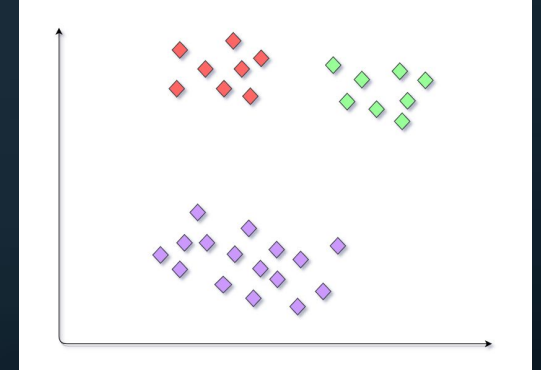
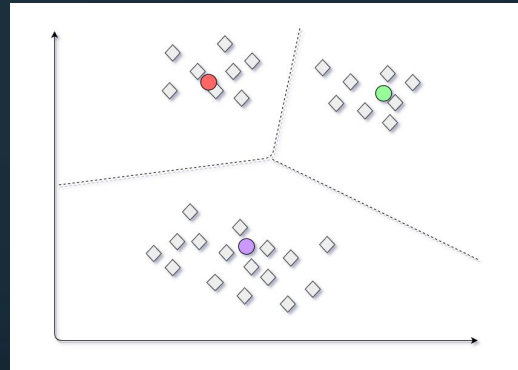
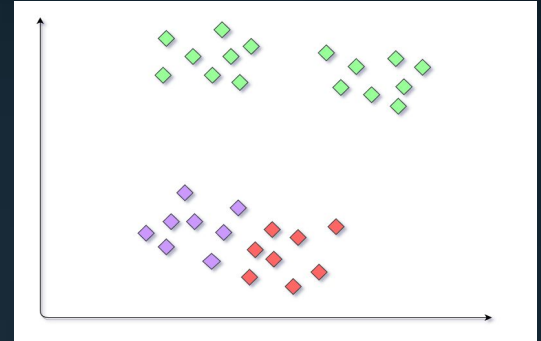
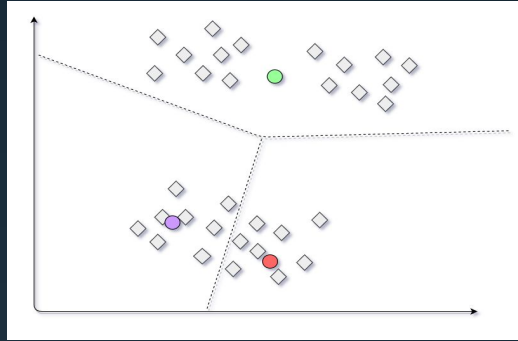




Random Initialization Trap in K-means Algorithm



We can do many random initialization to select best one



Random Initialization Trap Solution : K-means ++

- The more optimal the positioning of initial centroids, the fewer iterations will be required for convergence.
- **How does k means ++ work to solve the issue?**
 1. Randomly select the first centroid from the data points.
 2. For each data point compute its distance from the nearest, previously chosen centroid.
 3. Select the next centroid from the data points such that the probability of choosing a point as centroid is directly proportional to its distance from the nearest, previously chosen centroid. (i.e. the point having maximum distance from the nearest centroid is most likely to be selected next as a centroid)
 4. Repeat steps 2 and 3 until k centroids have been sampled



Random Initialization Trap Solution : K-means ++

sklearn.cluster.KMeans

```
class sklearn.cluster.KMeans(n_clusters=8, *, init='k-means++', n_init='warn', max_iter=300, tol=0.0001, verbose=0, random_state=None, copy_x=True, algorithm='lloyd')
```

[\[source\]](#)

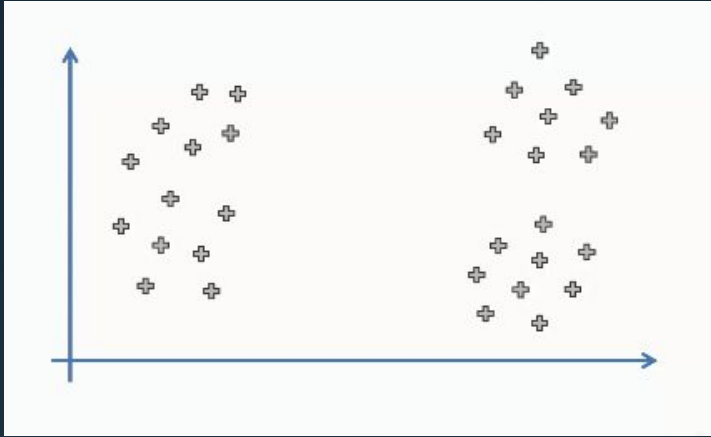
K-Means clustering.

init : {'k-means++', 'random'}, callable or array-like of shape (n_clusters, n_features), default='k-means++'

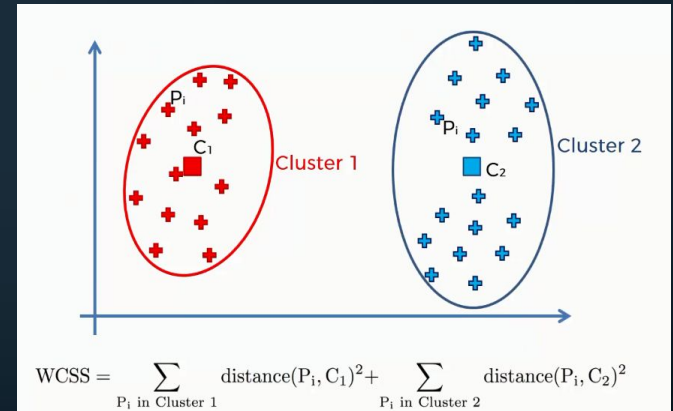
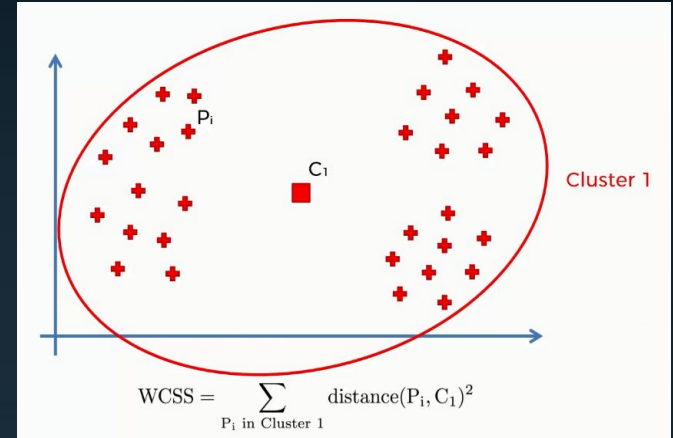
Method for initialization:



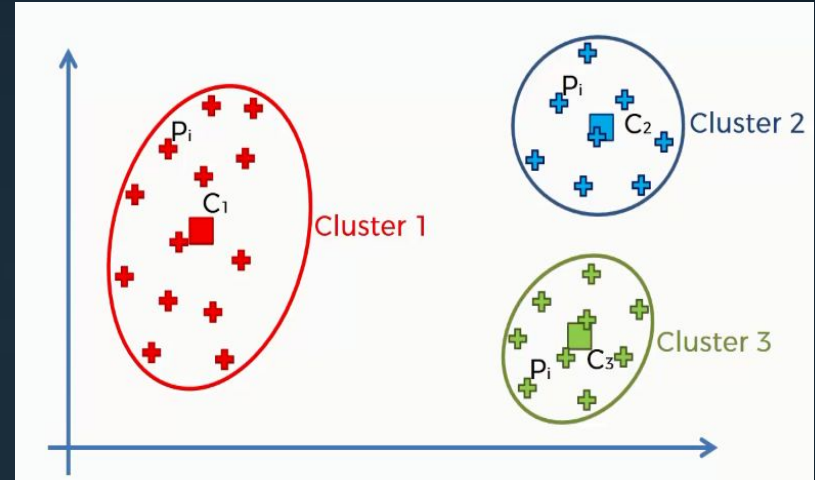
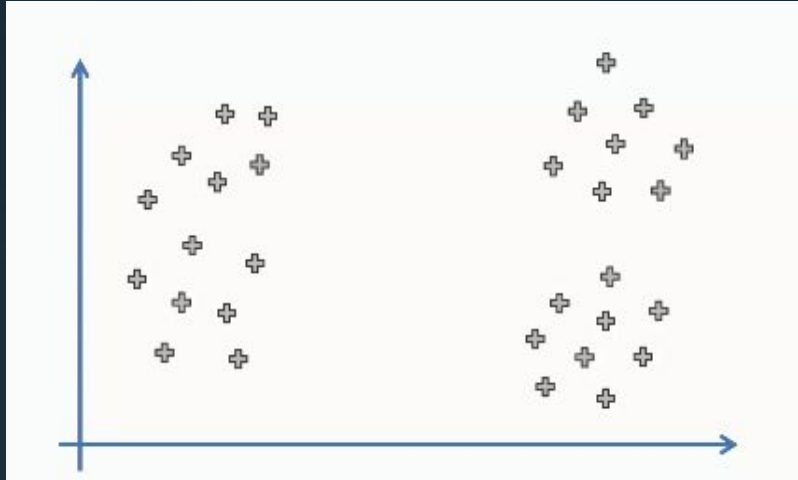
Elbow Method for optimal value of k



- WCSS is the sum of squares of the distances of each data point in all clusters to their respective centroids.
- It is also known as the inertia



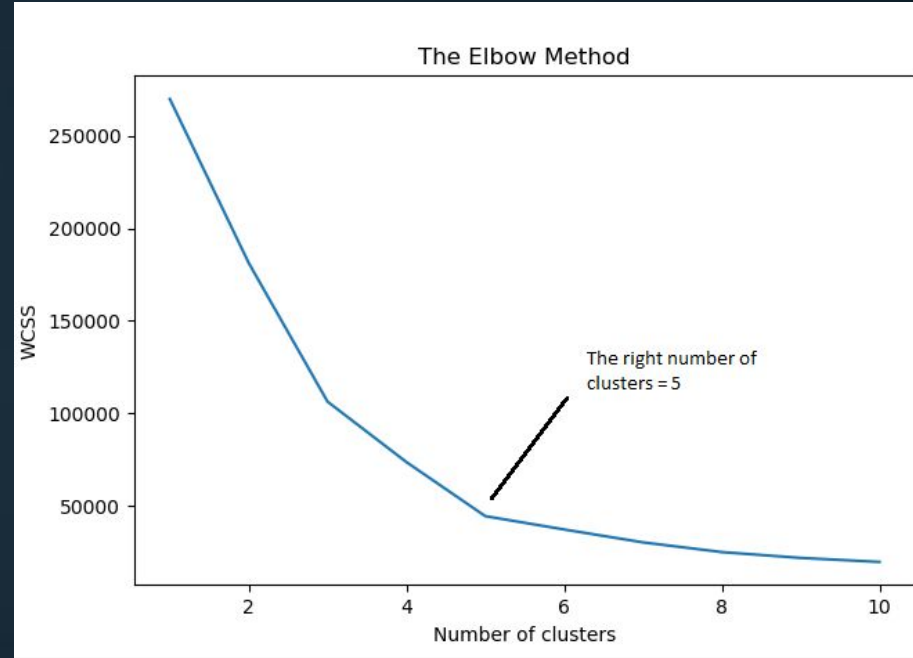
Elbow Method for optimal value of k



$$WCSS = \sum_{P_i \text{ in Cluster 1}} \text{distance}(P_i, C_1)^2 + \sum_{P_i \text{ in Cluster 2}} \text{distance}(P_i, C_2)^2 + \sum_{P_i \text{ in Cluster 3}} \text{distance}(P_i, C_3)^2$$

Elbow Method for optimal value of k

- k-means attempts to minimize the inertia (WCSS) when choosing Clusters
- More clusters means lower inertia
- Choose an "elbow" in the inertia plot where inertia begins to decrease more slowly

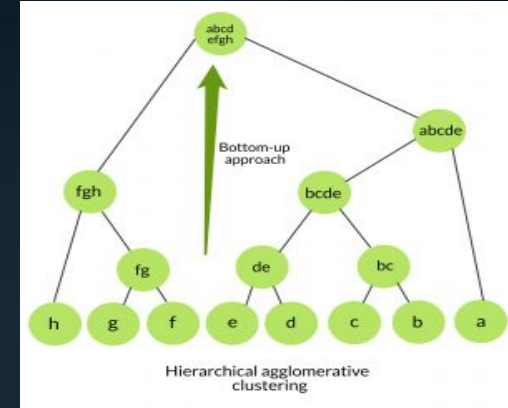


Hierarchical Algorithm

Hierarchical Clustering

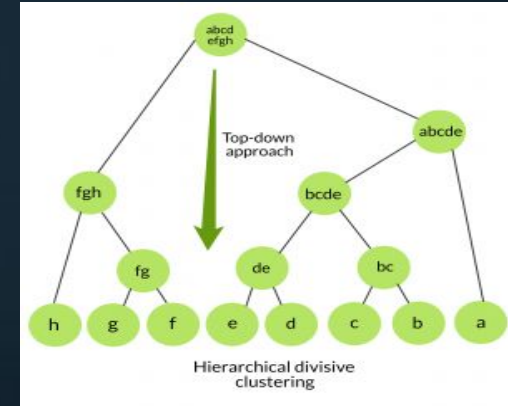
1. Agglomerative Clustering: Bottom-up approach

→ It treats each data point as a single cluster and then successively agglomerates pairs of clusters until all clusters have been merged into a single cluster that contains all data.

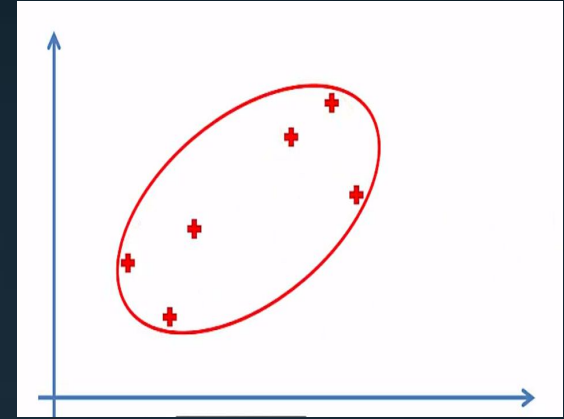
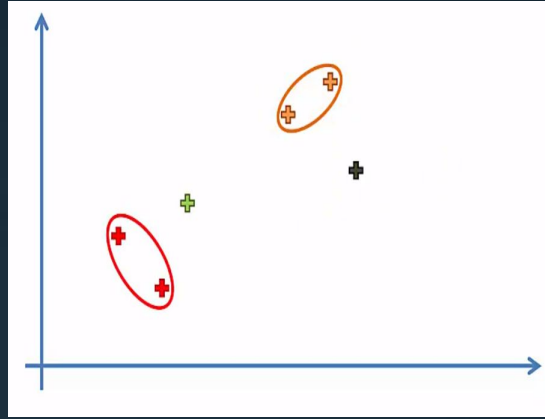
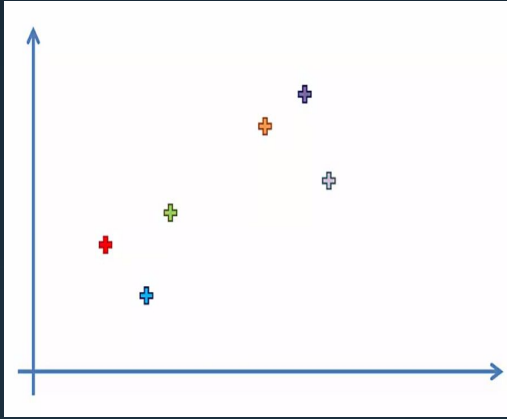


2. Divisive clustering: Top-down approach

→ It starts with a cluster that contains the whole data and proceeds by splitting cluster recursively until individual data have been splitted into single clusters.



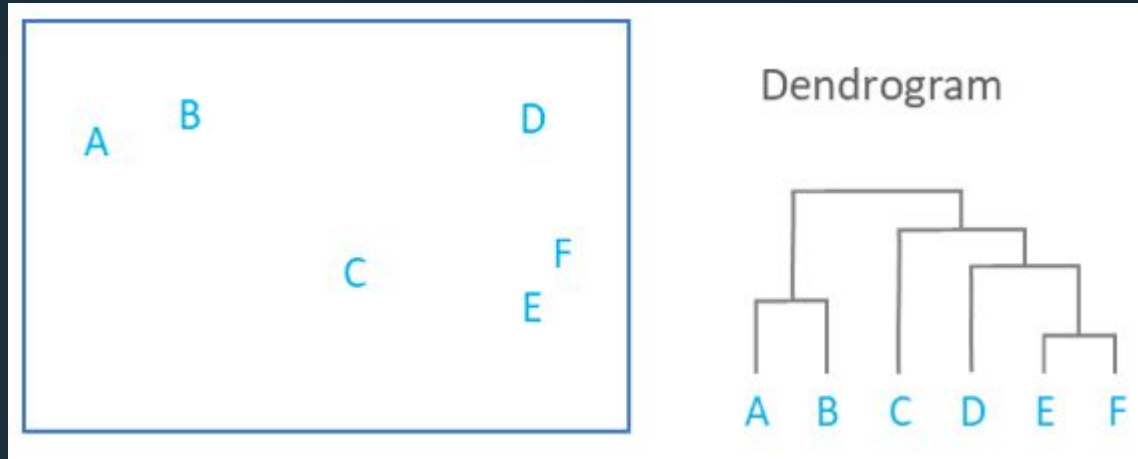
Agglomerative Hierarchical Clustering



1. Make each data point a single cluster.
2. Take the two closest *points* and make them one cluster.
3. Take the two closest *clusters* and make them one cluster.
4. Repeat till there is *one Cluster*.

Agglomerative Hierarchical Clustering: Dendrogram

- The way that HC agglomerative works is that it maintains memory of how we went to this process and that memory is stored in Dendrogram.



- Dendrogram is used to actually split the cluster into multiple clusters of related data points.

Agglomerative Hierarchical Clustering: Dendrogram

- The x-axis represents data points and y-axis represents the Euclidean distance between the clusters.
- Basically the horizontal line is a **threshold**, which defines the minimum distance required to be a separate cluster.

